



Ministère de l'Enseignement Supérieur et de la Recherche
Scientifique
Université de Carthage
Institut supérieur des technologies de l'information et de la
communication



Chapitre 2 : Les structures conditionnelles

Plan

1. Introduction
2. Structure conditionnelle simple (un seul choix)
3. Structure conditionnelle à deux choix
4. Structure conditionnelle imbriquée
5. Structure conditionnelle à multiple choix (selon)
6. Exercices

1. Introduction

Ce chapitre présente des actions exploitant les opérations de comparaison qui peuvent être effectuées par l'ordinateur. Deux actions sont présentées :

- L'action conditionnelle simple qui consiste à évaluer une condition (expression logique à valeur vrai ou faux) et effectuer le traitement relatif à la valeur de vérité trouvée.
- L'action conditionnelle à choix multiple qui consiste à évaluer une expression qui n'est pas nécessairement à valeur booléenne (elle peut avoir plus de deux valeurs) et selon la valeur trouvée, effectue un traitement.

2. Structure conditionnelle simple (un seul choix)

Contexte : Certaines instructions ne peuvent être exécutées que si une condition soit vérifiée.

Syntaxe :

En algorithmique

```
Si condition alors
|
|   <instructions>
|
Fin si
```

Traduction en langage C

```
if (condition)
{
    <instructions>
}
```

Explication : Avec le test simple, on associe une seule condition à un seul bloc de code. Si la condition est vraie, ce bloc est exécuté. Sinon, il n'est pas exécuté.

Exemples

Exemple 1 : Saisie des informations personnelles

- Saisie de la CIN
- Saisie du nom et du prénom
- Saisie de la situation familiale (Célibataire, Marié...)
- Saisie du nombre d'enfants ⇒ on ne peut pas exécuter cette instruction pour un célibataire.

Algorithme Informations

Variables

CIN, Nb_Enfants : entier

Nom, Prénom : chaîne

```

Situation : caractère
Début
Lire (CIN)
Lire (Nom, Prénom)
Lire (Situation)
Si (Situation <> 'C') alors
|   Lire (Nb_Enfants)
FinSi
Fin

```

Exemple 2 : calculer et afficher la valeur absolue d'un entier (sans utiliser la fonction ABS).

Algorithme fonction_absolue

```

Variables
X: entier
Début
écrire("donner un entier")
Lire (X)
Si (X < 0) alors
|    $X \leftarrow X * (-1)$ 
FinSi
Ecrire("valeur absolu de X est :", X)
Fin

```

Traduction en Langage C

```

#include<stdio.h>
void main()
{ int X;
  printf("donner un entier") ;
  scanf("%d", &X) ;
  if (X<0)
  X=X*(-1) ;
  printf("valeur absolu est
%d", X) ;
}

```

Remarque

- Si le bloc ne contient qu'une seule instruction, on peut omettre les accolades.
- Une erreur classique des programmeurs débutant consiste à placer un point-virgule juste après la condition, de la façon suivante :
- `if(x>0);`
`printf("La valeur de x est positive\n");`

Il ne s'agit pas d'une erreur de compilation, la syntaxe est correcte. En effet, le compilateur considère que le bloc du if est constitué d'une seule instruction, qui est... rien du tout. Donc, si $x > 0$, ici on ne fera rien de spécial. Cela signifie aussi que le printf ne se trouve pas dans le if, mais à l'extérieur. Et par conséquent, il sera exécuté quelle que soit la valeur de x.

3. Structure conditionnelle à deux choix

Contexte : Certaines instructions ne peuvent être exécutées que si une condition soit vérifiée et d'autres seront exécutées si la condition n'est pas vérifiée.

Syntaxe :

En algorithmique

```
Si condition alors
┌
│   <instructions 1>
│
│   Sinon
│       <instructions 1>
│
└
FinSi
```

Traduction en langage C

```
if (condition)
{<instructions 1>}
else
{<instructions 2>}
```

Explication : Si la condition est vraie, la suite d'instructions 1 est effectuée. Sinon, la suite d'instructions 2 est effectuée.

Exemple

Exemple de calcul de la valeur absolue pour être résolu en utilisant la structure conditionnelle à deux choix, comme suit :

Algorithme fonction_absolue

Variables

X: entier

Début

écrire("donner un entier")

Lire (X)

Si (X < 0) alors

┌ Ecrire("valeur absolu de X est :", -X)

Sinon

Ecrire("valeur absolu de X est :", X)

FinSi

Fin

Traduction en Langage C

```
#include<stdio.h>

void main()
{ int X;
  printf("donner un entier") ;
  scanf("%d", &X) ;
  if (X<0)
    printf("valeur absolu de %d est %d",X,-X) ;
  else
    printf("valeur absolu de %d est %d",X,X) ;
}
```

4. Structure conditionnelle imbriquée

Contexte : Une suite d'instructions d'une structure conditionnelle peut comporter d'autres instructions conditionnelles.

Syntaxe :

En algorithmique

```
Si condition 1 Alors
|
|   Si condition 2 Alors
|   |
|   |   <Suite d'instructions 1>
|   |
|   |   Sinon
|   |   |
|   |   |   <Suite d'instructions 2>
|   |   |
|   |   FinSi
|   |
|   |   Sinon
|   |   |
|   |   |   <Suite d'instructions 3>
|   |   |
|   |   FinSi
|
|   FinSi
```

Traduction en langage C

```
if (condition 1)
{
    if (condition 2)
    {
        <Suite d'instructions 1>
    }
    else
    {
        <Suite d'instructions 2>
    }
}
else
{
    <Suite d'instructions 3>
}
```

Remarque :

- Le nombre d'imbrication de structures conditionnelles n'est pas limité (selon le problème à résoudre).
- A chaque Si doit correspondre un FinSi

Exemple :

Ecrire un programme qui lit trois nombres entiers distincts et qui affiche leur maximum.

Algorithme maximum

Variables

X, Y, Z, max : entier

Début

écrire("donner trois entiers")

lire (X,Y,Z)

si (X>Y) et (X>Z) alors

| max ← X

FinSi

si (Y>X) et (Y>Z) alors

| max ← Y

FinSi

si (Z>X) et (Z>Y) alors

| max ← Z

FinSi

écrire("le maximum est :", max)

Fin

On peut remplacer ces trois structures conditionnelles simples par une structure conditionnelle imbriquée

si (X>Y) et (X>Z) alors

| max ← X

sinon

| si (Y>Z) alors

| | max ← Y

| sinon

| | si (Z>Y) alors

| | | max ← Z

| | finsi

| finsi

finsi

Traduction en langage C

```
#include <stdio.h>
int main()
{
    int X, Y, Z, max;
    printf("donner trois entier \n");
    scanf("%d", &X) ;
    scanf("%d", &Y) ;
    scanf("%d", &Z) ;
    if ((X>Y) && (X>Z))
        max=X;
    else
        if (Y>Z)
            max=Y;
        else
            if (Z>Y)
                max=Z;
    printf ("le maximum est %d", max);
}
```

5. Structure conditionnelle à choix multiple (selon)

Contexte : Dans le cas où la condition des structures conditionnelles Si imbriquées concernent la même variable, on peut utiliser alors la structure conditionnelle Selon. Le mot-clé `switch` reçoit entre parenthèses la variable à tester. Chaque valeur à comparer est ensuite indiquée dans le bloc du `switch`, en utilisant le mot-clé `case`, de la façon suivante :

Syntaxe

En algorithmique

Traduction en langage C

Selon (Variable) Faire

valeur1 : Suite d'instructions 1

valeur2 : Suite d'instructions 2

...

valeur n: Suite d'instructions n

Autre : Suite d'instructions 0

FinSelon

```
switch(variable)
{ case valeur1:
  suite d'instructions 1;
  break;
  case valeur2:
  suite d'instructions 2;
  break;
  .....
  case valeurn:
  suite d'instructions n;
  break;
  default:
  suite d'instructions 0;
}
```

Explication :

Le fonctionnement est le suivant. La variable spécifiée va être comparée successivement à chaque valeur proposée, jusqu'à ce qu'il y ait égalité. Si la variable n'est égale à aucune valeur, alors on sort simplement du switch sans rien faire de plus. Si on trouve une valeur égale, alors on exécute toutes les instructions situées en-dessous de la case correspondant. Attention : cela inclut non seulement les instructions de la case associée à la valeur, mais aussi celles de toutes les autres cases suivantes ! Or, il arrive fréquemment qu'on désire que seules les instructions placées directement en dessous du case correspondant soient exécutées, mais pas les suivantes. Pour arriver à ce comportement, il faut ajouter l'instruction `break` juste avant la case suivant

Exemple :

Écrire un algorithme qui lit le numéro d'un mois et qui affiche le numéro de son trimestre. (Janvier, Février, Mars : Trimestre 1...)

Solution 1 :

Algorithme Trimestre

Variables

Mois : Entier

Début

Écrire ("Donner le numéro du mois")

Lire (Mois)

Si (Mois=1 ou Mois=2 ou Mois=3) Alors

 Écrire ("Trimestre 1")

Sinon

Si (Mois=4 ou Mois=5 ou Mois=6) Alors

 Écrire ("Trimestre 2")

 sinon

Si (Mois=7 ou Mois=8 ou Mois=9) Alors

 Écrire ("Trimestre 3")

 Sinon

Si (Mois=10 ou Mois=11 ou Mois=12) Alors

 Écrire ("Trimestre 4")

 Sinon

 Écrire ("Numéro incorrect")

 FinSi

 FinSi

 FinSi

FinSi

Fin

Solution 2 :

Algorithme Trimestre

Variables

Mois : Entier

Début

Écrire ("Donner le numéro du mois")

Lire (Mois)

Selon (Mois) Faire

1, 2, 3 : Écrire ("Trimestre")

4, 5, 6 : Écrire ("Trimestre 2")

7, 8, 9 : Écrire ("Trimestre 3")

10, 11, 12 : Écrire ("Trimestre 4")

Autre : Écrire ("Numéro incorrect")

FinSelon

Fin

Traduction en langage C de la deuxième solution

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
    int mois;
```

```
    printf("donner numéro du mois");
```

```
    scanf("%d", &mois);
```

```
    switch(mois)
```

```
    {
```

```
        case 1 :
```

```
        case 2:
```

```
        case 3: printf("trimestre1"); break;
```

```
        case 4:
```

```
        case 5:
```

```
        case 6: printf("trimestre2"); break;
```

```
        case 7:
```

```
        case 8:
```

```
        case 9: printf("trimestre3"); break;
```

```
        case 10:
```

```
        case 11:
```

```
        case 12: printf("trimestre4");break;
```

```
        default: printf("erreur") ;
```

```
    }
```

```
}
```

Exercice 1 :

- 1) Ecrire un programme en langage C qui permet de saisir un entier puis vérifier s'il est pair ou impair. Concernant l'entier pair, vérifier s'il est divisible par son chiffre d'unité.

- 2) Traduire l'algorithme en langage C

Exemple : 14 → c'est un entier pair.

25 → c'est un entier impair et divisible par son chiffre d'unité.

127 → c'est un entier impair et non divisible par son chiffre d'unité.

Algorithme_verif

Variables

X, U : entier

Début

écrire ("donner un entier")

lire(X)

si (X mod 2=0)

 écrire("entier pair")

sinon

 U ← X mod 10

 si (X mod U =0)

 écrire ("impair et divisible par son chiffre d'unité ")

 sinon

 écrire ("impair et non divisible par son chiffre d'unité ")

 finsi

finsi

Traduction en langage C

```
#include <stdio.h>
```

```
void main()
```

```
{
```

```
  int X,U;
```

```
  printf("donner un entier");
```

```
  scanf("%d",&X);
```

```
  if(X % 2==0)
```

```
    printf("%d est pair",X);
```

```
  else
```

```
  {
```

```
    U=X%10;
```

```
    if(X%U==0)
```

```
      printf("%d est impair et divisible par son chiffre d'unité %d", X, U);
```

```
    else
```

```
      printf("%d est impair et non divisible par son chiffre d'unité %d", X,U);
```

```
  }
```

```
}
```

Exercice 2 :

- 1) Ecrire un algorithme permettant de résoudre une équation du second degré $ax^2 + bx + c = 0$. Avec a, b et c sont des valeurs lu à partir du clavier
- 3) Traduire l'algorithme en langage C

Algorithme équation

Variables

a, b, c, delta : entier

X1, X2 : réel

début

écrire ("donner des valeurs pour a, b et c ")

lire (a, b, c)

Si (a = 0) alors

 écrire ("équation du premier degré ")

Sinon

 delta ← b*b - 4*a*c

 Si (delta > 0) alors

 X1 ← (-b - sqrt(delta)) / (2*a)

 X2 ← (-b + sqrt(delta)) / (2*a)

 écrire ("les solutions de l'équation sont ", X1, X2)

 Sinon

 Si (delta = 0) alors

 écrire ("solution est :", -b / (2*a))

 Sinon

 écrire ("pas de solutions réelles ")

 Fin si

 Fin si

Fin si

Fin

Traduction en langage C

```
#include <stdio.h>
```

```
#include <math.h>
```

```
int main()
```

```
{
```

```
    int A, B, C, D;
```

```
    printf("Introduisez les valeurs pour A, B, et C : ");
```

```
    scanf("%d %d %d", &A, &B, &C);
```

```
    if (A==0)
```

```
    {
```

```
        printf(" équation du premier degré :");
```

```
    }
```

```
    else
```

```
    {
```

```
        D = pow(B,2) - 4.0*A*C;
```

```
if (D>0)
{
printf("Les solutions réelles de cette équation sont :\n");
printf(" x1 = %f", (-B+sqrt(D))/(2*A));
printf(" x2 = %f", (-B-sqrt(D))/(2*A));
}
else
if (D==0)
{
printf("Cette équation a une seule solution réelle :\n");
printf(" x = %f", (float)-B/(2*A));
}
else
printf("Cette équation n'a pas de solutions réelles.\n");
}
```